

ليه الـ Query اتنفذت مرتين؟

السر اللي أغلب مطوري #C ميعرفهوش عن Deferred Execution

"كنت فاكّر إن LINQ بتننفذ أول ما بكتبها. . .

لحد ما اكتشفت إن نفس الـ Query راحت للـ Database مرتين."

دي كانت رحلتي لفهم واحدة من أهم الأفكار التي اتبنت عليها LINQ.

Deferred Execution

لو فهمتها. . . هتبدأ تشوف LINQ بطريقة مختلفة تمامًا.

خلينا نبدأ بتجربة صغيرة.

```
var numbers = Enumerable.Range(1, 5);

var evenNumbers = numbers.Where(x =>
{
    Console.WriteLine($"Filtering {x}");

    return x % 2 == 0;
});

Console.WriteLine("Query Created");
```

قبل ما تشغل الكود. . .

توقع معايا.

ايه الـ Output؟

معظم الناس هتتوقع حاجة زي:

```
Filtering 1
Filtering 2
Filtering 3
```

Filtering 4

Filtering 5

Query Created

لأننا كتبنا :

Where(...)

صح؟

لكن... لما تشغل البرنامج...

هتشفوف :

Query Created

بس.

ولا سطر واحد من :

Filtering ...

وده أول لغز في رحلتنا.

فين الـ Where؟

هو مش المفروض يفلتر؟

ليه الكود اللي جوه الـ Lambda مشغلش أصلاً؟

هل فيه Bug؟

هل الـ Compiler تجاهل الكود؟

ولا فيه حاجة أعمق بتحصل؟

خلي السؤال ده في دماغك.

لأن الإجابة عليه هتغير الطريقة اللي بتفكر بيها في LINQ بالكامل.

قبل ما أعرف يعني إيه **Deferred Execution**...

كنت مقتنع إن LINQ بتشتغل بنفس الطريقة اللي معظم الدوال في #C بتشتغل بيها.

يعني لما أكتب:

```
DoSomething();
```

فالم Method بتتنفذ فورًا.

ولما أكتب:

```
var now = DateTime.Now;
```

القيمة بتتجاب في نفس اللحظة.

فطبيعي جدًا كنت أتوقع إن:

```
var evenNumbers = numbers.Where(x => x % 2 == 0);
```

هيحصل فيها الآتي.



يعني كنت فاكّر إن `Where()` هتلف على كل العناصر.

وتفلترها.

وترجع Collection جديدة جاهزة للاستخدام.

وده في الحقيقة... منطقي جدًا.

لأن ده اللي بيحصل مع معظم الـ APIs اللي بنتعامل معاها.

لكن خلينا نجرب حاجة بسيطة.

ضيف السطر ده بعد إنشاء ال Query مباشرة.

```
Console.WriteLine("Query created successfully.");
```

وبعدين بعده مباشرة.

```
Console.WriteLine("Program finished.");
```

هيبقى الكود بالشكل ده.

```
var numbers = Enumerable.Range(1, 5);

var evenNumbers = numbers.Where(x =>
{
    Console.WriteLine($"Filtering {x}");

    return x % 2 == 0;
});

Console.WriteLine("Query created successfully.");

Console.WriteLine("Program finished.");
```

وشغل البرنامج.

النتيجة؟

```
Query created successfully.
```

```
Program finished.
```

ولا سطر واحد من:

```
Filtering...
```

وده معناه إن ال Lambda اللي كتبناها...

متنفذتش أصلاً.

مش إنها اتنفذت بسرعة.

ومش إنها رجعت Collection فاضية.

هي ببساطة...

متفدثش.

وده يخليك تسأل سؤال مهم جدًا.

لو `Where()` ما نفذتش... يبقى إيه اللي رجعتة؟

يعني إيه المتغير ده؟

```
var evenNumbers = ...
```

لو مش List... ومش Array...

ومفیش Filtering حصل...

يبقى هو إيه؟

وده كان أول سؤال خلاني أفتح Documentation الخاصة بـ LINQ.

ولقيت إن كل المشكلة بدأت من افتراض واحد.

كنت فاكّر إن `Where()` بترجع النتيجة.

لكن الحقيقة...

هي بترجع **وصفة لتحضير النتيجة لاحقًا**.

وده هو أول خيط في رحلتنا لفهم **Deferred Execution**.# 🔍 The Investigation

لحد دلوقتي...

إحنا اكتشفنا حاجتين.

الأولى...

الكود ده:

```
var evenNumbers = numbers.Where(x =>
{
    Console.WriteLine($"Filtering {x}");

    return x % 2 == 0;
});
```

منفذش الـ Lambda.

والثانية...

المتغير `evenNumbers` موجود بالفعل.

يعني `Where()` رجعت شيئاً ما.

لكنها لم ترجع النتيجة.

طيب...

رجعت إيه؟

أول حاجة عملتها

فتحت Visual Studio.

وحطيت Breakpoint بعد السطر ده.

```
var evenNumbers = numbers.Where(x => x % 2 == 0);
```

ولما وقفت على المتغير...

لقيت المفاجأة.

مش List.

ومش Array.

ومفيش حتى عنصر واحد متخزن.

كان نوعه:

```
System.Linq.Enumerable+WhereEnumerableIterator<int>
```

وده كان أول دليل.

استنى...

يعني إيه Iterator؟

في البداية...

الاسم كان غريب.

لكن بعد شوية قراءة...

بدأت أفهم الفكرة.

الـ Iterator مش Collection.

هو Object صغير...

وظيفته إنه يعرف إزاي يجيب العنصر التالي لما تطلبه.

لاحظ الكلمة.

لما تطلبه.

مش دلوقتي.

تخيل الموقف ده

أنت داخل مطعم.

طلبت بيتزا.

الجارسون كتب الطلب.

وقالك:

"تمام... الطلب اتسجل."

هل البيتزا اتعملت؟

لا.

هل دخلت الفرن؟

لا.

كل اللي حصل...

إن الطلب بقى موجود.

ولما يبجي دورك...

المطبخ بيدأ يشتغل.

وده تقريباً اللي LINQ بتعمله.

لما تكتب:

```
var evenNumbers = numbers.Where(x => x % 2 == 0);
```

هي مش بتلف على العناصر.

ولا بتفلترها.

ولا بتعمل List.

هي فقط بتقول:

"لو حد طلب العناصر بعد كده..."

اعمل Filtering باستخدام الشرط ده."

يعني Where () بترجع إيه؟

الإجابة الأدق هي:

بتبني خطة (Execution Plan) لكيفية الحصول على البيانات، لكنها لا تنفذها.

وده سبب إنك تقدر تكتب:

```
var query =  
    numbers  
        .Where(x => x % 2 == 0)  
        .OrderBy(x => x)  
        .Take(3)  
        .Select(x => x * 10);
```

رغم إنك كتبت أربع عمليات...

إلا إن ولا واحدة منهم اشتغلت لحد اللحظة دي.

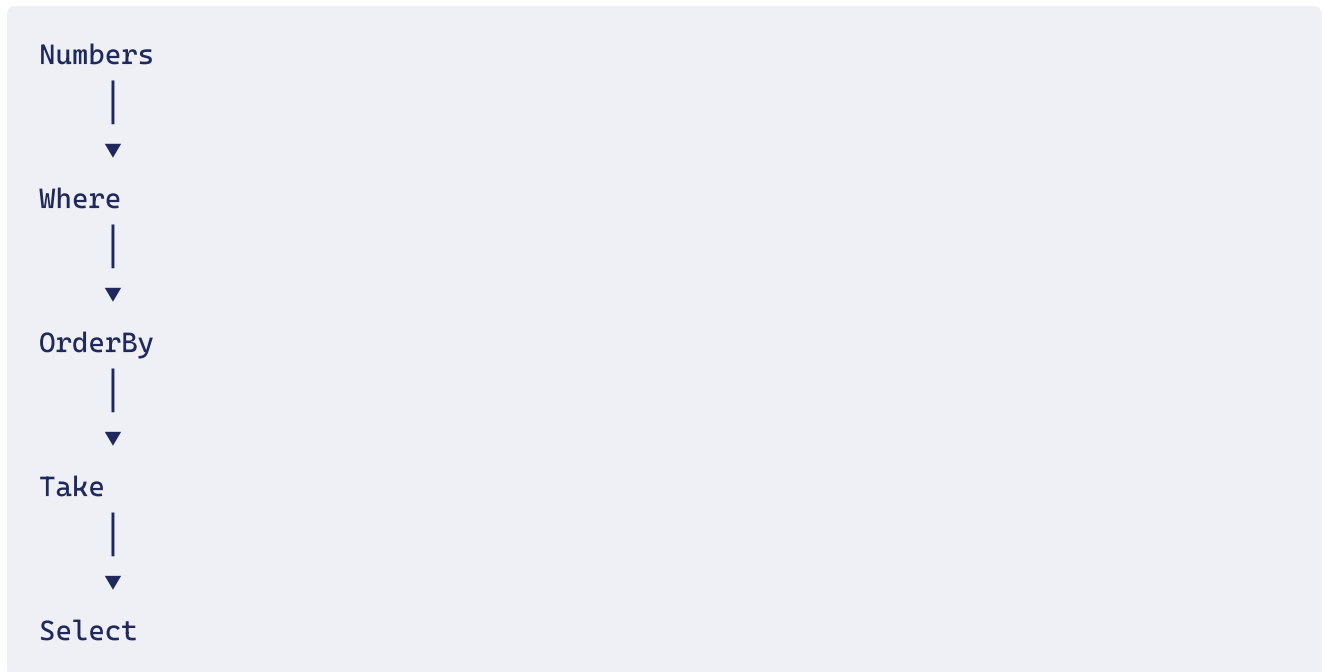
كل اللي حصل...

إن LINQ بنت Pipeline.

تخيلها بالشكل ده

بدل ما تتخيل LINQ كأنها بتشغل خطوة بخطوة...

تخليها كأنها بتبني خط إنتاج.



لكن...

الخط ده كله واقف.

مفيش ولا عنصر اتحرك.

مفيش ولا شرط اتنفذ.

مفيش ولا عملية Sort حصلت.

لسه.

وده هو السبب الحقيقي وراء الاسم.

Deferred Execution

يعني:

"تنفيذ مؤجل."

مش تنفيذ بطيء.

ومش تنفيذ في Background.

ومش Async.

هو فقط...

مؤجل إلى أن يحتاج البرنامج النتيجة فعلاً.

ودي واحدة من أهم الأفكار اللي بُنيت عليها LINQ.

طيب امتى بتنفذ؟

لحد اللحظة دي...

إحنا عرفنا إن `Where()` مَبْتَنَفَذْش.

وعرفنا إنها بترجع Query جاهزة للتنفيذ لاحقاً.

لكن...

فيه سؤال لسه بدون إجابة.

إمتى التنفيذ بيبدأ؟

مين اللي بيقول لـ LINQ:

"يلا... ابدأ اشتغل."

خلينا نجرب.

التجربة

نفس الكود.

```
var numbers = Enumerable.Range(1, 5);

var evenNumbers = numbers.Where(x =>
{
    Console.WriteLine($"Filtering {x}");

    return x % 2 == 0;
});
```

لحد هنا... عارفين إن مفيش حاجة حصلت.

لكن...

ضيف السطر ده.

```
foreach (var number in evenNumbers)
{
    Console.WriteLine(number);
}
```

وشغل البرنامج.

الـ Output هيكون.

```
Filtering 1
Filtering 2
2
Filtering 3
Filtering 4
4
Filtering 5
```

لاحظ حاجة غريبة جدًا.

البرنامج مش عمل:

```
Filtering 1
Filtering 2
Filtering 3
Filtering 4
Filtering 5

2
4
```

وده اللي أغلب الناس متوقعاه.

لكن اللي حصل هو.

```
Filtering 1

Filtering 2

2

Filtering 3

Filtering 4
```

4

Filtering 5

يعني...

الفلتر والطباعة كانوا يحصلوا بالتبادل.

عنصر... بعنصر.

لو LINQ كانت بتنفذ مرة واحدة...

كان الطبيعي يحصل الآتي.

numbers

↓

Filter All

↓

Create New Collection

↓

foreach prints

لكن ده مش اللي حصل.

اللي حصل أقرب لكده.

foreach

↓

Give me next item

↓

Where checks item

↓

If accepted

↓

Return item

↓

Print item

↓

Repeat

لاحظ.

مفيش Collection جديدة اتعملت .

ولا كل العناصر اتفلترت مرة واحدة .

البرنامج كان بيتعامل مع كل عنصر لوحده .

مين بدأ التنفيذ؟

الإجابة هي .

foreach

مش Where () .

ولا Select () .

ولا OrderBy () .

ال foreach هي أول حد قال :

"أنا محتاج أول عنصر ."

ولما خلص منه . . .

قال .

"هات اللي بعده."

وبعدين.

"هات اللي بعده."

وهكذا.

Conveyor Belt تخیل

تخیل عندك خط إنتاج.

Numbers

↓

Where

↓

Select

↓

Take

↓

foreach

كل محطة مستتية.

محدث شغال.

أول ما **foreach** تطلب أول عنصر...

بيبدأ الخط كله يتحرك.

لكن...

عنصر واحد فقط.

مش كل العناصر مرة واحدة.

وهنا تظهر قوة Deferred Execution

لأن البرنامج مش محتاج يفلتر مليون عنصر .

قبل ما يبدأ يشتغل .

هو بيحيب عنصر .

يفلتره .

يرجعه .

وبعدين ينتقل للي بعده .

وده اسمه .

Streaming Execution

وده أحد الأسباب الرئيسية اللي بتخلي LINQ تستهلك Memory أقل في سيناريوهات كتير .

طيب ... هل foreach هي الوحيدة؟

لا .

أي Method تحتاج النتيجة الفعلية ...

هتبدأ التنفيذ .

زي .

`ToList()`

أو .

`ToArray()`

أو .

`First()`

أو .

Single()

أو .

Count()

أو .

Any()

كل واحدة من دول محتاجة تعرف النتيجة .

فبتقول لـ LINQ :

"كفاية تجهيز . . .

ابدأ التنفيذ ."

Mental Model

قبل ما أفهم Deferred Execution . . .

كنت متخيل إن الكود بيمشي بالشكل ده .

Where()

↓

Execute

↓

Result

لكن الحقيقة .

Where()

↓

Build Query

↓

Wait...

↓

foreach

↓

Execute

↓

Return First Item

↓

Return Second Item

↓

...

Until Finished

وده سبب إن LINQ مش بتشتغل أول ما تكتبها. . .

هي بتشتغل **أول ما تحتاجها فعلاً**.



The Production Bug

لحد دلوقتي. . .

كل الأمثلة اللي شوفناها كانت على:

```
List<int>
```

وده جميل.

لكن. . .

المشكلة الحقيقية ظهرت أول مرة لما كنت شغال مع Entity Framework Core.

كان عندي الكود ده .

```
var activeUsers = context.Users
    .Where(u => u.IsActive);

foreach (var user in activeUsers)
{
    Console.WriteLine(user.Name);
}

Console.WriteLine("-----");

foreach (var user in activeUsers)
{
    Console.WriteLine(user.Name);
}
```

الكود شكله طبيعي جدًا .

وأغلبنّا كتب حاجة شبيهه قبل كده .

لكن...

لما فتحت SQL Profiler ...

لقيت المفاجأة .

```
SELECT *
FROM Users
WHERE IsActive = 1
```

وبعدها بثنائي...

نفس الـ Query اتنفذت مرة ثانية .

```
SELECT *
FROM Users
WHERE IsActive = 1
```

استنتي...

أنا معملتش Where مرتين .

ولا كتبت Query جديدة .

أول تفسير جه في دماغي

قلت يمكن EF Core بتعمل Cache.

أو يمكن SQL Server.

أو يمكن فيه Bug.

لكن...

بعد ما رجعت للـ Documentation...

اكتشفت إن المشكلة في الكود نفسه.

السبب الحقيقي

خلينا نفكر حاجة اتعلمناها من شوية.

```
var activeUsers =  
    context.Users.Where(...);
```

السطر ده...

مش جاب البيانات.

هو بس بنى Query.

ولسه مستني حد يقول له:

"هاتلي النتائج."

وأول **foreach** عملت كده.

```
foreach (var user in activeUsers)
```

ف EF Core ترجمت الـ LINQ إلى SQL.

وبعتها للـ Database.

ورجعت النتائج.

بعد ما ال foreach خلصت ...

كل حاجة انتهت.

وبعدين ...

جت ال foreach الثانية.

```
foreach (var user in activeUsers)
```

وبصت على ال Query.

وقالت.

"هاتلي النتائج."

ف EF Core نفذت ال Query ...

مرة ثانية.

لأنها معندهاش نتائج مخزنة.

هي عندها **طريقة للحصول على النتائج**.

وده فرق مهم جدًا.

تخيلها بالشكل ده

تخيل إن عندك ريموت.

كل مرة تدوس الزرار.

التلفزيون يشتغل.

الريموت نفسه ...

مش بيخزن صورة.

هو مجرد وسيلة للوصول إليها.

وده تقريبًا دور ال Query.

هي مش البيانات.

هي وسيلة للحصول على البيانات.

طب إمتي المشكلة دي تبقى خطيرة؟

تخيل إن الـ Query دي بترجع .

100 ألف Record .

أو بتاخذ ثانيتين .

أو عليها خمس Join .

كل مرة تعمل .

foreach

أنت بتقول لـ EF Core .

"روح اسأل الـ Database مرة ثانية. "

ولو أنت مش واخذ بالك . . .

ممكن الـ API تبقى أبطأ بمرتين .

أو ثلاث مرات .

بدون أي سبب واضح .

الحل؟

لو أنت عارف إنك هتستخدم النتائج أكثر من مرة . . .

اعمل Materialization مرة واحدة .

```
var activeUsers = context.Users
    .Where(u => u.IsActive)
    .ToList();
```

اللي حصل دلوقتي .

Database



Query Executed Once

↓

List<User>

↓

foreach

↓

foreach

ال Database اتسألت مرة واحدة فقط.

لأنك دلوقتي معاك النتائج فعلاً.

مش مجرد Query.

مهم جداً

لاحظ إني **مش بقول استخدم ToList()** دائماً.

لأن ده هيضيع ميزة كبيرة من مزايا LINQ.

أنا بقول.

استخدمها . . .

لما تكون محتاج تعيد استخدام نفس النتائج أكثر من مرة.

وده فرق كبير.

طوبى ليه متتفدش فوري؟

بعد ما شوفنا إن ال Deferred Execution ممكن تسبب مشاكل . . .

زي إن ال Query تتنفذ مرتين . . .

ممكن ييجي في دماغك سؤال منطقي.

طوبى ليه Microsoft عملتها كده من الأساس؟

ليه متتفدش ال Query أول ما أكتب:

```
var evenNumbers = numbers.Where(x => x % 2 == 0);
```

وترجعلي List وخلص؟

السؤال ده ممتاز .

ولإجابة عليه . . .

خلينا نبص على مثال بسيط .

المثال

```
var result = numbers
    .Where(x => x > 10)
    .OrderBy(x => x)
    .Select(x => x * 2)
    .Take(5);
```

لو LINQ كانت بتنفذ فورًا . . .

فاللي هيحصل تقريبًا هيكون كده .

numbers

↓

Where

↓

List جديدة

↓

OrderBy

↓

List جديدة

↓

Select

↓

List جديدة

↓

Take

↓

List جديدة

يعني بعد كل خطوة...

هتنشئ Collection جديدة.

وننسخ فيها البيانات.

وده مع عدد كبير من العناصر هيستهلك:

- أكثر Memory.
- وقت أكثر.
- أكثر Garbage Collection.

وده مش أفضل تصميم.

اللي LINQ بتعمله

بدل ما تنفذ كل خطوة لوحدها...

هي بتجمع كل الخطوات الأول.

Where

↓

OrderBy

↓

Select

↓

Take

وبعدين . . .

لما تحتاج البيانات فعلاً . . .

تنفذ الـ Pipeline مرة واحدة.

وده معناه:

- مرور أقل على البيانات.
- استهلاك Memory أقل.
- كود أكثر مرونة.

مثال عملي

تخيل عندك مليون رقم.

وأنت عايز أول 5 أرقام زوجية فقط.

```
var result = numbers
    .Where(x => x % 2 == 0)
    .Take(5);
```

بفضل الـ Deferred Execution . . .

.مش هتفضل تكمل لحد آخر المليون LINQ.

أول ما تلاقي خمس عناصر . . .

هنتوقف.

وده يوفر وقت كبير.

الخلاصة

الـ Deferred Execution مش Bug.

ومش قرار غريب .

هي طريقة ذكية بتخلي LINQ :

- تنفذ أقل شغل ممكن .
- تستهلك Memory أقل .
- وتنفذ فقط عند الحاجة .

المشكلة مش في الـ Deferred Execution .

المشكلة لما نستخدمها . . .

من غير ما نبقى عارفين إنها موجودة .

ال Best Practices

بعد كل اللي شفناه . . .

إيه الحاجات اللي ممكن تعملها علشان تستفيد من Deferred Execution وتتجنب مشاكلها؟

1. متستخدمش `()ToList` بشكل تلقائي

من الأخطاء الشائعة إن بعض المطورين يضيفوا `()ToList` بعد أي Query بدون سبب .

```
var users = context.Users
    .Where(u => u.IsActive)
    .ToList();
```

لو هتستخدم البيانات مرة واحدة فقط . . .

فالك `()ToList` غالبًا مش هتضيف أي قيمة، بل هتجبر LINQ على تنفيذ الـ Query وتحميل كل النتائج في الذاكرة .

استخدمها فقط عندما تحتاج النتائج نفسها، وليس مجرد الـ Query .

2. لو هتستخدم النتائج أكثر من مرة... اعمل `Materialize`

```
var users = context.Users
    .Where(u => u.IsActive)
    .ToList();
```

```
foreach (var user in users)
```

```
{
    // ...
}

foreach (var user in users)
{
    // ...
}
```

بدلاً من تنفيذ نفس الـ Query مرتين، هتتنفذ مرة واحدة فقط، وبعدها هتتعامل مع البيانات الموجودة في الذاكرة.

3. اعرّف إمّتى بتتعامل مع Query... وإمّتى بتتعامل مع Data

فيه فرق كبير بين الاتنين.

```
IQueryable<User>
```

أو

```
IEnumerable<User>
```

غالبًا بيمثلوا **طريقة للحصول على البيانات**.

أما:

```
List<User>
```

فدي **البيانات نفسها**.

كل ما تميز بين الاتنين، هتقل الأخطاء المتعلقة بالـ Deferred Execution.

4. راقب الـ SQL في EF Core

لو بتستخدم Entity Framework Core...

شغل Logging أو SQL Profiler من وقت للتاني.

أوقات كتير هتكتشف إن نفس الـ Query بتتنفذ أكثر من مرة بدون ما تكون واخد بالك.

5. متعبرش Deferred Execution مشكلة

هي مش Bug.

هي Feature.

لكن أي Feature لازم تفهمها قبل ما تستخدمها.

References

- Microsoft Learn — LINQ (Language Integrated Query)
- Microsoft Learn — Deferred Execution and Lazy Evaluation
- .NET Runtime Source Code (`System.Linq`)
- C# Language Specification
- Eric Lippert's Blog